100 DAYS OF MACHINE LEARNING

DAY 3

Complete Notes, Worked Examples & Practice Workbook

Official CampusX playlist — Videos 8, 9 and 10

Video 8: Applications of Machine Learning
Video 9: Machine Learning Development Life Cycle
Video 10: Data Engineer vs Data Analyst vs Data Scientist vs ML Engineer

DAY 3 STUDY SNAPSHOT	COMPLETION
Official videos completed	3 videos
Approximate video time	29 + 25 + 26 ≈ 80 minutes
Quiz result	10/10 (100%)
Main outcome	Connect real-world ML problems, the ML lifecycle, and data/ML job roles

How to use this workbook: Read the concept notes, work through the examples, try the exercises before viewing the answers, and finish with the revision checklist. English explanations include occasional Roman Urdu cues for easier understanding.

Beginner note: examples in this workbook are for learning. A real ML system requires validated data, appropriate evaluation, privacy safeguards, and human oversight where decisions affect people.

Learning Objectives

- Identify common real-world ML applications and map them to suitable task types.
- Explain the Machine Learning Development Life Cycle (MLDLC) from problem framing to monitoring.
- Understand data preparation, EDA, feature engineering, model training, evaluation, deployment, and maintenance.
- Distinguish the common responsibilities of Data Engineers, Data Analysts, Data Scientists, and ML Engineers.
- Use a small Python file to explore forecasting, recommendations, anomaly flags, text classification baselines, and data checks.

Video 8 — Applications of Machine Learning

Machine Learning (ML) is useful when a system can learn patterns from examples and use those patterns to make predictions, rankings, groupings, or decisions. The right approach depends on the goal, available data, and cost of mistakes.

1. Common applications and task types

Area / example	Possible ML task	Illustrative inputs	Output / goal
E-commerce recommendations	Recommendation / ranking	User interactions, item features, past purchases	Rank items a user may find relevant
Sales or demand planning	Forecasting / regression	Past sales, season, promotions, prices	Estimate future demand
Bank transaction monitoring	Anomaly detection / classification	Transaction amount, timing, merchant, account patterns	Flag unusual activity for review
Customer feedback	Text classification / sentiment	Review text, labels from examples	Estimate positive, negative, or other categories
Medical image support	Image classification / detection	Images with appropriately reviewed labels	Assist trained professionals with image analysis
Manufacturing maintenance	Classification / anomaly detection	Sensor readings, vibration, temperature, fault history	Flag potential equipment problems
Customer segmentation	Clustering	Behavior and purchase summaries	Group customers with similar patterns
Navigation and transport	Prediction / optimization	Traffic, road network, time, historical trips	Estimate arrival times or support route choices

Important: an application does not determine one unique algorithm. The task and evaluation requirements matter. For example, a bank might combine rules, anomaly detection, and supervised classification rather than rely on one model.

2. How to choose a task type

Task type	Question it answers	Simple example
Regression	What numeric value should we estimate?	Predict next month's sales
Classification	Which category is most likely?	Classify a support ticket as billing or technical
Clustering	Which examples are similar, without predefined labels?	Group customers by behavior
Recommendation / ranking	What should be shown first to this user?	Rank films for a viewer
Anomaly detection	Which examples are unusually different?	Flag a sudden unusual sensor reading
Forecasting	What may happen over time?	Estimate weekly demand from historical demand

Roman Urdu cue: pehle business sawal clear karo. Phir decide karo ke jawab number, category, group, ranking, ya unusual-event alert hona chahiye.

Video 8 — A Practical Application Workflow

Step	What happens	Example: student-support application
1. Define goal	Translate a vague idea into a measurable problem.	Estimate future academic support needs; do not treat a prediction as a final judgment about a student.
2. Collect data	Gather relevant, permitted, sufficiently representative examples.	Use appropriately authorized study-hours, attendance, and past academic records.
3. Inspect and prepare	Check missing values, duplicates, units, invalid values, and leakage.	Confirm CGPA scale, attendance range, and whether each feature was known at prediction time.
4. Train and evaluate	Compare a baseline with candidate models on data not used for fitting.	Compare errors on a held-out set and inspect performance across relevant groups.
5. Deploy carefully	Integrate a validated model into the workflow.	Show a prediction with uncertainty and a clear explanation of limitations.
6. Monitor	Track input changes, errors, reliability, and real-world outcomes.	Check whether later cohorts differ from historical training data.

A prediction is not automatically a causal explanation. If students who study more also have higher CGPA in a dataset, that association alone does not prove study time caused the difference; other factors may be involved.

Video 9 — Machine Learning Development Life Cycle (MLDLC)

The MLDLC is the repeatable process used to turn a real-world problem into an ML system and keep it useful after release. It is not always a straight line: evaluation can send the team back to data preparation, feature engineering, or even problem framing.

3. Lifecycle stages in detail

Stage	Key activities	Questions to ask
1. Problem framing	Define users, decision, target, constraints, and success metric.	What decision will the output support? What is the cost of a false prediction?
2. Data gathering	Find relevant sources; verify permission, quality, and coverage.	Does the data represent the people and conditions where the system will be used?
3. Data preparation / wrangling	Fix formats, duplicates, missing values, inconsistent labels, and invalid entries.	Are cleaning choices justified and documented? Could they introduce bias?
4. Exploratory Data Analysis (EDA)	Study distributions, relationships, outliers, and target balance with summaries and plots.	What patterns or data issues need investigation?
5. Feature engineering and selection	Transform raw data into useful inputs; remove irrelevant or leaking features.	Would this feature be available at the moment of prediction?
6. Train candidate models	Create suitable train/validation/test splits; fit a baseline and candidates.	Is the split appropriate for time, users, or repeated records?
7. Evaluate and select	Use metrics suited to the task; compare against a baseline and error costs.	Does the model generalize to unseen data and meet the real requirement?
8. Deploy	Integrate the model with an app, API, batch process, or workflow.	Can it run reliably, securely, and within latency/cost limits?
9. Monitor and maintain	Track data drift, model performance, failures, fairness, and cost; update or roll back if needed.	Has the real-world data or behavior changed?

Roman Urdu memory line: Masla samjho → data jama karo → data saaf aur explore karo → useful features banao → model train/test karo → deploy karo → performance monitor karo. Zaroorat par pichlay steps dobara repeat hote

hain.

MLDLC — Important Concepts and Worked Example

4. Data leakage, splits, baselines, and metrics

Concept	Meaning	Why it matters
Baseline	A simple reference method, such as predicting the training-set average for a numeric target.	A complex model should beat a sensible simple approach.
Train set	Data used to fit model parameters.	The model learns patterns from these examples.
Validation set	Data used to compare options or tune hyperparameters.	Repeatedly using the test set for tuning makes the final evaluation less trustworthy.
Test set	Held-out data used for a final evaluation after choices are made.	It approximates performance on unseen examples when the split is appropriate.
Data leakage	Information unavailable at prediction time, or information from the target/test data, improperly enters training.	Can make results look unrealistically good.
Overfitting	The model learns training-specific noise or detail.	Training performance can be strong while new-data performance is weak.
Data drift	Input data or relationships change over time.	A deployed model may become less reliable and needs monitoring.

Example: building a CGPA prediction prototype

Goal: estimate a future numeric CGPA-related outcome for learning purposes. First, define exactly which term or future outcome is the target; do not mix current CGPA with a future CGPA without a clear time boundary.

Lifecycle step	Concrete action
Problem framing	Define target, prediction time, intended user, error metric (for example, MAE), and limitations.
Data gathering	Collect authorized historic records and document what each column means.
Data preparation	Validate CGPA values against the correct scale; investigate missing values and duplicates.
EDA	Check target distribution, sample size, relationships, and possible subgroup differences.
Feature engineering	Use only information available before the prediction point, such as previous-term GPA or recorded attendance.
Training/evaluation	Use a suitable split; if predicting future terms, consider a time-aware split. Compare with a simple baseline.
Deployment	Present a clearly labeled estimate with limitations; keep an appropriate human decision process.
Monitoring	Compare later predictions with outcomes; inspect error, drift, and whether use is causing unintended harm.

5. Example metrics

Problem	Possible metrics	Caution
Regression	MAE, RMSE	A low average error can hide large errors for some groups or cases.
Classification	Precision, recall, F1, ROC-AUC; accuracy where appropriate	With rare events, accuracy alone can be misleading.
Ranking / recommendation	Precision@K, Recall@K, NDCG, user-centered evaluation	Offline ranking metrics may not fully predict user satisfaction.
Forecasting	MAE, RMSE, or task-appropriate scaled metrics	Evaluate on later time periods, not randomly mixed future data.

Video 10 — Understanding Data and ML Job Roles

Teams divide work so that data can be collected reliably, understood, modeled, and delivered as a dependable product. Job titles vary by organization, and responsibilities often overlap.

Role	Main focus	Typical work / tools	Example deliverable
Data Engineer	Reliable data infrastructure and pipelines	Build ETL/ELT jobs, data ingestion, storage, orchestration, data quality checks; SQL, Python, cloud/data platforms	A tested pipeline that supplies clean, timely data
Data Analyst	Describe what happened and why it may have happened	SQL queries, spreadsheets, dashboards, reporting, KPI analysis, communicating findings	A sales dashboard and evidence-based business insights
Data Scientist	Investigate patterns, test hypotheses, and build predictive/experimental solutions	Statistics, EDA, feature engineering, experiments, model development, evaluation; Python/SQL	An evaluated model or analysis that informs a decision
ML Engineer	Make ML systems work reliably in production	Model serving, APIs, pipelines, monitoring, testing, latency/cost/reliability, retraining workflows	A deployed model service with monitoring and rollback plans

6. How the roles collaborate

Stage of a project	Likely contributors
Data sources and reliable pipelines	Data Engineer; also domain teams and source-system owners
Business metrics and reporting	Data Analyst; product/business stakeholders
Exploration, experiments, and model development	Data Scientist; subject-matter experts
Production integration, serving, and reliability	ML Engineer; software, platform, and data engineering teams
Ongoing decisions and feedback	All relevant roles plus users, product owners, and governance/privacy stakeholders

7. Example: a student CGPA insights project

- **Data Engineer:** creates a reliable, permissioned pipeline for approved academic records and checks schema/quality.
- **Data Analyst:** builds a dashboard showing historical grade distributions and cohort trends.
- **Data Scientist:** tests hypotheses and evaluates whether historical features can predict a clearly defined future target.
- **ML Engineer:** packages a validated model into a secure service, adds monitoring, and plans rollback/retraining.

These are typical patterns, not strict rules. In a small team one person may perform several roles; in a large organization each may be specialized.

8. B2C vs B2B context

Term	Meaning	Example
B2C — Business to Consumer	A company serves individual consumers directly.	A shopping app recommends products to a shopper.
B2B — Business to Business	A company provides a product or service to another organization.	An analytics platform helps a retailer forecast inventory.

Practice Lab — Python Examples

Download the separate **CampusX_ML_Day_03_Practice.py** file and run it with Python 3. It uses only the Python standard library, so no pip installation is needed. The code demonstrates simple baselines rather than claiming to be production ML.

9. What the Python file demonstrates

Function	Concept practiced	Limitation
moving_average_forecast	Forecast a next value using recent numeric observations	Assumes the recent mean is useful; not a full time-series model
classify_review	Map review text to sentiment labels using word lists	Rule-based baseline; not trained ML and misses context/negation
flag_anomalies	Flag values far from a mean using a standard-deviation rule	Simple rule; may fail on skewed data or small samples
recommend_top_items	Rank items by matching tags	Toy content-based score; no learned user/item representations
clean_student_records	Validate required fields and numeric ranges	Illustrative checks only; deletion can bias data if used without investigation
predict_cgpa_baseline	Produce a transparent toy estimate using a nearest previous-GPA record	Not trained, validated, or suitable for real academic decisions
role_guide	Print a concise job-role comparison	Role descriptions are simplified and organizations differ

10. How to run and experiment

```
python CampusX_ML_Day_03_Practice.py
```

Try these changes after the script runs:

- Change the sales list and compare the moving-average forecast with the actual next value (if known).
- Add a review with both positive and negative words and observe how the simple rule behaves.
- Change the anomaly threshold; notice how the number of flagged values can change.
- Add movie tags and change user preference tags to see the ranking change.
- Add a missing or out-of-range student record and observe how the quality checks handle it.
- Read each function's docstring and explain why its output should not be treated as a reliable production prediction.

11. Exercises — attempt before reading the answer key

Exercise 1. A video platform wants to order films for each viewer. Which task type fits best, and why?

Exercise 2. A model gets 99% accuracy on training data but 68% on held-out data. Name one likely issue and two steps to investigate.

Exercise 3. Your CGPA dataset includes a column recorded only after the final exam, but the prediction must be made before the exam. What is the risk?

Exercise 4. Match these deliverables to likely roles: (a) production model API and monitoring; (b) executive KPI dashboard; (c) dependable data ingestion pipeline; (d) evaluated experiment and predictive model.

Exercise 5. Why is monitoring still needed after an ML model is deployed?

Answer Key, Quiz Review & Revision Checklist

12. Worked answers

Exercise 1: Recommendation/ranking. The system estimates which films are most relevant to a viewer and orders them accordingly.

Exercise 2: Overfitting or a train/test distribution mismatch may be present. Check split design, leakage, data quality, model complexity, and performance on a suitable validation set; compare against a baseline.

Exercise 3: Data leakage. That column would not be available at prediction time, so using it would make evaluation unrealistically optimistic and would not match real use.

Exercise 4: (a) ML Engineer; (b) Data Analyst; (c) Data Engineer; (d) Data Scientist—while acknowledging that responsibilities can overlap.

Exercise 5: Inputs and relationships may change; systems can fail; accuracy, fairness, latency, and costs can shift. Monitoring helps detect issues and decide whether to investigate, retrain, or roll back.

13. Day 3 quiz review

You reported a score of **10/10 (100%)** on the combined Videos 8–10 quiz. The quiz tested application/task matching, problem framing, EDA, feature engineering, overfitting, role responsibilities, and post-deployment monitoring.

14. Quick revision checklist

- ■ I can map a business question to regression, classification, clustering, recommendation/ranking, forecasting, or anomaly detection.
- ■ I can explain why the goal and success metric must be defined before model selection.
- ■ I can list the lifecycle: problem framing → data gathering → preparation → EDA → feature engineering → training → evaluation → deployment → monitoring.
- ■ I can explain train, validation, and test sets, and why data leakage is dangerous.
- ■ I know why a baseline is useful and why the evaluation metric must match the task.
- ■ I can explain overfitting, data drift, and the need for monitoring.
- ■ I can distinguish typical Data Engineer, Data Analyst, Data Scientist, and ML Engineer responsibilities.
- ■ I can run the Python file and describe what each demonstration does and does not prove.

15. Key vocabulary

Term	Plain-language meaning
Feature	An input variable used by a model.
Target / label	The outcome the model is trained to estimate or classify.
EDA	Exploring data using summaries, tables, and visualizations before modeling.
Feature engineering	Creating or transforming useful input variables.
Data leakage	Using information that would not legitimately be available for model training or prediction.
Deployment	Making a model available in a real workflow or application.
Monitoring	Checking a live system's data, performance, failures, and operational behavior.
Drift	A change in input data or the relationship between inputs and outcomes over time.

Day 3 takeaway: A successful ML project is more than choosing an algorithm. It starts with a well-defined problem, relies on appropriate data and evaluation, fits into a real workflow, and is monitored after release. Teams need data

infrastructure, analysis, modeling, and production engineering to work together.

Learning resource: official CampusX 100 Days of Machine Learning playlist. This workbook is an original study aid and does not replace the videos or professional guidance.